

■ PYTHON SET METHODS — COLOURFUL HANDWRITTEN NOTES

Classroom + Real-Time IT Examples + Practice Labs

1. What is a Set?

Definition

A **set** is an unordered collection of **unique** values in Python. Sets are written using `{ }`. A set automatically removes duplicate values.

```
servers = {"web01", "web02", "db01", "web01"}  
print(servers) # duplicate web01 is removed
```

Real-time use

Sets are very useful for removing duplicate data, checking membership quickly, comparing server lists, user permissions, IP addresses, tags and security findings.

2. Important Set Properties

Property	Meaning	Example
Unique	Duplicates are removed	$\{1,2,2,3\} \rightarrow \{1,2,3\}$
Unordered	No fixed index position	<code>s[0]</code> ■
Mutable	Can add/remove values	<code>s.add(10)</code>
No indexing	Cannot use normal indexing	<code>s[0]</code> ■
Mixed types	Can store different hashable types	<code>{'web01', 443, True}</code>

3. Core Set Methods — Add / Remove

add()

Adds one item to a set.

```
servers = {"web01", "db01"}  
servers.add("web02")  
print(servers)
```

Real-time use

Real-time: Add a newly discovered server or resource.

update()

Adds multiple items from another iterable.

```
servers = {"web01", "web02"}  
new_servers = ["db01", "cache01"]  
servers.update(new_servers)  
print(servers)
```

Real-time use

Real-time: Merge a new inventory list into an existing server set.

remove()

Removes an item. Raises KeyError if the item does not exist.

```
ports = {80, 443, 8080}
ports.remove(8080)
print(ports)
```

Real-time use

Real-time: Remove a retired port/resource when you know it exists.

discard()

Removes an item if present; no error if absent.

```
ports = {80, 443}
ports.discard(8080)
print(ports)
```

Real-time use

Real-time: Safely remove an optional resource.

pop()

Removes and returns an arbitrary set element.

```
servers = {"web01", "db01", "cache01"}
removed_server = servers.pop()
print(removed_server)
print(servers)
```

Real-time use

Real-time: Consume one item when exact removal order is not important.

clear()

Removes all elements from the set.

```
active_hosts = {"web01", "web02"}
active_hosts.clear()
print(active_hosts) # set()
```

Real-time use

Real-time: Clear a temporary inventory or processing set.

copy()

Creates a shallow copy of the set.

```
prod_servers = {"web01", "web02"}  
backup_servers = prod_servers.copy()  
print(backup_servers)
```

Real-time use

Real-time: Create a working copy before changing a set.

4. Set Mathematical / Comparison Methods ■

union()

Combines values from both sets.

```
prod = {"web01", "web02"}  
dr = {"web02", "web03"}  
all_servers = prod.union(dr)  
print(all_servers)
```

Real-time use

Useful when combining two inventories without duplicates.

intersection()

Returns values common to both sets.

```
prod = {"web01", "web02", "db01"}  
critical = {"db01", "db02"}  
common = prod.intersection(critical)  
print(common) # {'db01'}
```

Real-time use

Useful for finding common servers, users, permissions or security findings.

difference()

Returns values present in the first set but not the second.

```
old = {"web01", "web02", "db01"}  
new = {"web01", "web02"}  
removed = old.difference(new)  
print(removed) # {'db01'}
```

Real-time use

Useful for detecting resources removed from the latest inventory.

symmetric_difference()

Returns values that exist in either set, but not both.

```
team_a = {"AWS", "Azure", "Docker"}  
team_b = {"Azure", "Kubernetes", "Docker"}  
different = team_a.symmetric_difference(team_b)  
print(different)
```

Real-time use

Useful for finding differences between two environments, teams or configurations.

issubset()

Checks whether every item in one set exists in another.

```
required = {"HTTPS", "SSH"}
allowed = {"HTTP", "HTTPS", "SSH"}
print(required.issubset(allowed)) # True
```

Real-time use

Useful for checking whether required permissions/features are included.

issuperset()

Checks whether a set contains every item of another set.

```
allowed = {"HTTP", "HTTPS", "SSH"}
required = {"HTTPS", "SSH"}
print(allowed.issuperset(required)) # True
```

Real-time use

Useful for validating access policies or capability sets.

isdisjoint()

Checks whether two sets have no common elements.

```
prod = {"web01", "db01"}
test = {"web10", "db10"}
print(prod.isdisjoint(test)) # True
```

Real-time use

Useful for checking whether two inventories, IP pools or resource groups overlap.

5. Set Operators — Faster Syntax

```
a = {"web01", "web02"}
b = {"web02", "db01"}

print(a | b) # union
print(a & b) # intersection
print(a - b) # difference
print(a ^ b) # symmetric difference
```

Remember

| = UNION & = INTERSECTION - = DIFFERENCE ^ = SYMMETRIC DIFFERENCE

6. Membership Check

```
allowed_users = {"admin", "vishwa", "devops"}

username = "vishwa"

if username in allowed_users:
    print("Access allowed")
else:
    print("Access denied")
```

Real-time security example

Sets are excellent for membership checks such as allowed users, blocked IPs, approved ports, IAM permissions and security tags.

7. Real-Time DevOps / Cloud Examples

Find New Servers

```
old_inventory = {"web01", "web02", "db01"}
new_inventory = {"web01", "web02", "db01", "cache01"}

new_servers = new_inventory.difference(old_inventory)
print(new_servers) # {'cache01'}
```

Find Removed Servers

```
old_inventory = {"web01", "web02", "db01"}
new_inventory = {"web01", "web02"}

removed_servers = old_inventory.difference(new_inventory)
print(removed_servers) # {'db01'}
```

Compare IAM Permissions

```
required = {"read_s3", "read_logs", "read_ec2"}
granted = {"read_s3", "read_logs"}

missing = required.difference(granted)
print(missing) # {'read_ec2'}
```

Remove Duplicate IPs

```
ip_addresses = ["10.0.0.1", "10.0.0.2", "10.0.0.1", "10.0.0.3"]
unique_ips = set(ip_addresses)
print(unique_ips)
```

8. Set vs List vs Tuple

Feature	Set	List	Tuple
Duplicates	■ No	■ Yes	■ Yes
Ordered	■ No fixed order	■ Yes	■ Yes
Mutable	■ Yes	■ Yes	■ No
Indexing	■ No	■ Yes	■ Yes
Main use	Unique data / comparison	Changing collection	Fixed collection

9. Interview Questions ■

- What is a set in Python?
- Why does a set remove duplicates?
- Can we use indexing with a set? — No.
- Difference between `remove()` and `discard()`?
- Difference between `union()` and `intersection()`?
- What is the difference between `difference()` and `symmetric_difference()`?
- What is the use of `issubset()` and `issuperset()`?
- Why are sets useful for membership checks?
- How do you remove duplicates from a list? — Convert it to a set.
- How do you create an empty set? — `set()`, not `{}` because `{}` creates an empty dictionary.

10. Practice Labs ■

- Create a set of 10 server names and add two new servers.
- Create two sets of ports and find common ports using `intersection()`.
- Compare old and new server inventories and find newly added servers.
- Compare two IAM permission sets and find missing permissions.
- Create a list with duplicate IP addresses and remove duplicates using `set()`.
- Use `issubset()` to check whether required permissions are granted.
- Use `isdisjoint()` to verify two environments have no common servers.
- Practice `remove()`, `discard()`, `pop()` and `clear()` and observe their behavior.

11. Quick Reference — Set Methods

Method	Purpose
<code>add(x)</code>	Add one item
<code>update(x)</code>	Add multiple items
<code>remove(x)</code>	Remove item; error if absent
<code>discard(x)</code>	Remove item; no error if absent
<code>pop()</code>	Remove and return arbitrary item
<code>clear()</code>	Remove all items
<code>copy()</code>	Copy set
<code>union()</code>	Combine unique values
<code>intersection()</code>	Common values
<code>difference()</code>	Only values in first set
<code>symmetric_difference()</code>	Values in either, but not both
<code>issubset()</code>	Check subset
<code>issuperset()</code>	Check superset
<code>isdisjoint()</code>	Check no overlap

■ Remember

Set = unique data + fast membership + powerful comparison.

Core methods include add, update, remove, discard, pop, clear, copy, union, intersection, difference, symmetric_difference, issubset, issuperset and isdisjoint.